PROJET MÉDIATHÈQUE · EXERCICES JAVA

4 exercices Java autour du projet

De l'adhérent au service d'emprunt : objets, Streams, règles métier, exceptions

4	49	6	2h20
exercices	vérifications	règles métier	environ



Slah Hassine
Formateur · Tech Lead Java · 14 ans d'expérience

Adherent · Catalogue · PolitiqueEmprunt · ServiceEmprunt
Fichiers à compléter avec vérifications automatiques

Mode d'emploi

Quatre exercices progressifs, tous tirés du projet de la Médiathèque. Chaque exercice est **un seul fichier Java** qui contient le code à compléter et des **vérifications automatiques** : tu sais tout de suite si ta réponse est juste.

Où faire les exercices ? Dans un projet à part

Ne mets pas ces fichiers dans **mediatheque-api** : ils sont dans un projet séparé, **mediatheque-exercices**, déjà prêt. Pourquoi :

- mediatheque-api est le **vrai projet** : il ne doit contenir que le code de l'application (et son historique Git doit rester propre).
- Chaque exercice est **autonome** : il a son propre main, ses propres classes simplifiées et ses vérifications. Ce n'est pas le code définitif.
- Les bonnes solutions passeront ensuite dans le vrai projet, au bon endroit (le paquet domain), avec de vrais tests JUnit.

À savoir · Garde les deux projets ouverts dans deux fenêtres IntelliJ : mediatheque-api pour la formation, mediatheque-exercices pour t'entraîner.

Le squelette est prêt · Tu n'as rien à créer : le projet `mediatheque-exercices` contient déjà les 4 fichiers à compléter, le réglage du JDK 25 et une configuration de lancement par exercice. Ton formateur te donne le lien GitHub pour le télécharger en zip.

Lien GitHub du projet :	

Étape 1 · Télécharger le projet en zip

1	Ouvre le lien GitHub du projet dans ton navigateur.
2	Clique sur le bouton vert **Code**, puis sur **Download ZIP**.
3	Décompresse le zip dans le même dossier que mediatheque-api (par exemple `~/dev`).
4	Le dossier obtenu s'appelle `mediatheque-exercices-main` : renomme-le en `mediatheque-exercices`.
5	IntelliJ : **File › Open…**, choisis le dossier `mediatheque-exercices`, puis **Open** et **Trust Project**. Choisis **New Window** pour garder mediatheque-api ouvert à côté.

Étape 2 · Vérifier que tout est en place

1	Dans le panneau Project : les 4 fichiers `Exercice1` à `Exercice4` sont dans `src`, sans souligné rouge.
2	En haut à droite, la liste déroulante propose **Exercice 1 - Adherent** à **Exercice 4 - Service emprunt**.
3	Si un bandeau jaune affiche **Project JDK is not defined** : clique sur **Setup SDK** et choisis le **JDK 25** (ou File › Project Structure › Project › SDK).

> **Conseil ·** Pas d'accès à GitHub ? Le PDF suffit : crée un projet Java dans IntelliJ, puis pour chaque exercice clic droit sur `src` › **New › Java Class** avec le nom exact du fichier, et colle le code de départ puis le code de l'annexe.

Étape 3 · Lancer un exercice

1	En haut à droite, choisis **Exercice 1 - Adherent** dans la liste et clique sur la **flèche verte** à côté.
2	Autre façon : ouvre `Exercice1_Adherent.java` et clique sur la **flèche verte** dans la marge, à gauche de `public static void main`, puis **Run 'Exercice1_Adherent.main()'**.
3	Le résultat s'affiche dans la fenêtre **Run** en bas : au départ, tout est en [A FAIRE].
4	Après chaque modification, relance : **Ctrl + R** sur Mac, **Shift + F10** sur Windows (ou la flèche verte en haut à droite).

Astuce IntelliJ	Mac	Windows	À quoi ça sert
Voir tous les TODO	View › Tool Windows › TODO	View › Tool Windows › TODO	La liste de tout ce qu'il reste à faire, fichier par fichier
Compléter le code	Ctrl + Espace	Ctrl + Espace	Propose les méthodes : tape livres.stream(). puis Ctrl + Espace
Reformater	Cmd + Alt + L	Ctrl + Alt + L	Remet l'indentation au propre
Corriger une erreur	Alt + Entrée	Alt + Entrée	Sur un souligné rouge : propose une correction (import manquant…)

Sans IntelliJ, dans un terminal (Java 21 ou plus) :

```
cd mediatheque-exercices/src
java Exercice1_Adherent.java
```

Lire le résultat

Affichage	Signification
[A FAIRE]	La méthode n'est pas encore écrite (elle contient encore le TODO).
[ERREUR]	La méthode est écrite mais le résultat n'est pas celui attendu : relis l'énoncé de la question.
[OK]	La vérification passe.
Score : 11 / 11 Bravo !	L'exercice est terminé.

Règles du jeu

- Remplace chaque `throw new UnsupportedOperationException("À compléter");` par ton code. Le commentaire TODO juste au-dessus donne un indice.
- Ne modifie pas la partie « VÉRIFICATIONS » en bas du fichier.
- Fais les questions dans l'ordre et relance le fichier après chaque question.
- Java 21 ou plus suffit (le JDK 25 de la formation convient).

Les 4 exercices

N°	Exercice	Notions	Durée	Vérif.
1	L'adhérent	Programmation objet · encapsulation · validation	30 min	11
2	Le catalogue	Streams · Optional · records	30 min	9
3	La politique d'emprunt	Règles R1 à R5 · exceptions · java.time · BigDecimal	35 min	16
4	Le service d'emprunt	Synthèse · objets · Streams · Optional · exceptions	45 min	13

À savoir · Ce document contient **tout** le nécessaire : l'énoncé, le code de départ de chaque fichier et, en annexe, le code des vérifications. Sans accès au projet GitHub, recopie le code de départ puis l'annexe dans un fichier portant le nom indiqué.

1

PROGRAMMATION OBJET · ENCAPSULATION · VALIDATION

L'adhérent

Contexte · La médiathèque enregistre ses adhérents. Un adhérent ne doit **jamais** exister avec des données invalides : c'est lui-même qui vérifie ses données dans son constructeur. C'est le principe du **modèle riche** vu en séance 2.

Ce que tu dois écrire

Q	Méthode	Comportement attendu
Q1	Le constructeur	Refuser un nom ou un prénom vide (null, "" ou seulement des espaces) et un email sans @ ou sans point, avec une `IllegalArgumentException`. Enregistrer les valeurs sans espaces autour, l'email en minuscules. Un nouvel adhérent est actif.
Q2	`getNomComplet()`	Renvoyer « Prénom NOM », par exemple « Alice DURAND ».
Q3	`desactiver()` / `reactiver()`	Changer l'état actif de l'adhérent (un adhérent inactif ne peut plus emprunter : règle R3).
Q4	`estNouveau(aujourdhui)`	Vrai si l'adhérent est inscrit depuis **moins de 7 jours**. À 7 jours exactement, il n'est plus nouveau.
Q5	`equals()` et `hashCode()`	Règle R6 : l'email est unique. Deux adhérents sont égaux s'ils ont le même email. Toujours redéfinir les deux méthodes ensemble.

Coups de pouce

- Pour tester « vide ou seulement des espaces » : `isBlank()` (Java 11).
- Pour compter des jours entre deux dates : `ChronoUnit.DAYS.between(debut, fin)`.
- Dans equals, `if (!(o instanceof Adherent autre)) return false;` teste le type et crée la variable en une ligne.

Ce que les vérifications testent

Ton code doit faire passer chacune de ces vérifications :

Q	Vérification
Q1	un adhérent valide est créé et actif
Q1	les espaces sont retirés et l'email est en minuscules
Q1	nom vide refusé
Q1	prénom null refusé
Q1	email sans @ refusé
Q2	nom complet = « Alice DURAND »
Q3	désactiver puis réactiver
Q4	inscrit il y a 6 jours → nouveau
Q4	inscrit il y a 7 jours → plus nouveau (limite exacte)
Q5	même email → égaux, même hashCode
Q5	emails différents → pas égaux

Terminé quand · le fichier `Exercice1_Adherent.java` affiche **Score : 11 / 11 Bravo !**

Le code de départ · `Exercice1_Adherent.java`

Remplace chaque `throw new UnsupportedOperationException("À compléter");` par ton code. Le TODO juste au-dessus donne l'idée.

Exercice1_Adherent.java · à compléter

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.Objects;
import java.util.function.BooleanSupplier;

/**
 * EXERCICE 1 · L'adhérent (programmation objet, encapsulation, validation)
 *
 * Contexte : la médiathèque enregistre ses adhérents. Un adhérent ne doit jamais être
 * créé avec des données invalides : c'est lui-même qui vérifie ses données (modèle riche).
 *
 * Lancer : java Exercice1_Adherent.java
 * Objectif : toutes les vérifications à [OK].
 */
public class Exercice1_Adherent {

    static class Adherent {

        public static final int PERIODE_NOUVEL_ADHERENT_JOURS = 7;

        private final String nom;
        private final String prenom;
        private final String email;
        private final LocalDate dateInscription;
        private boolean actif;

        /**
         * Q1. Le constructeur refuse les données invalides (IllegalArgumentException) :
         *  - nom ou prénom null ou vide (ou seulement des espaces) → "Le nom est
obligatoire" / "Le prénom est obligatoire"
```

Exercice1_Adherent.java · à compléter

```
         *  - email null, sans @ ou sans point → "Email invalide"
         * Il enregistre le nom et le prénom sans espaces autour, l'email sans espaces et
en minuscules.
         * Un nouvel adhérent est actif.
         */
        Adherent(String nom, String prenom, String email, LocalDate dateInscription) {
            // TODO : vérifie chaque paramètre (isBlank, contains), puis affecte les
attributs (strip, toLowerCase)
            throw new UnsupportedOperationException("À compléter");
        }

        public String getNom() { return nom; }
        public String getPrenom() { return prenom; }
        public String getEmail() { return email; }
        public LocalDate getDateInscription() { return dateInscription; }
        public boolean isActif() { return actif; }

        /** Q2. Renvoie « Prénom NOM », par exemple « Alice DURAND ». */
        public String getNomComplet() {
            // TODO : concatène le prénom, un espace et le nom en majuscules
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q3. Désactive l'adhérent (il ne pourra plus emprunter : règle R3). */
        public void desactiver() {
            // TODO : une seule ligne
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q3 bis. Réactive l'adhérent. */
        public void reactiver() {
            // TODO : une seule ligne
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q4. Un adhérent est « nouveau » s'il est inscrit depuis MOINS de 7 jours. */
        public boolean estNouveau(LocalDate aujourdhui) {
            // TODO : ChronoUnit.DAYS.between(dateInscription, aujourdhui)
            throw new UnsupportedOperationException("À compléter");
        }

        /**
         * Q5. Règle R6 : l'email est unique. Deux adhérents sont donc égaux s'ils ont le
même email.
         * Redéfinir equals ET hashCode (toujours les deux ensemble).
         */
        @Override
        public boolean equals(Object o) {
            // TODO : même référence → true ; instanceof Adherent autre → comparer les
emails
            throw new UnsupportedOperationException("À compléter");
        }

        @Override
        public int hashCode() {
            // TODO : Objects.hash(...) sur le même attribut que equals
            throw new UnsupportedOperationException("À compléter");
        }
```

Exercice1_Adherent.java · à compléter

```
        @Override
        public String toString() {
            return "Adherent[" + getNomComplet() + ", " + email + (actif ? "" : ",
inactif") + "]";
        }
    }
```

2

STREAMS · OPTIONAL · RECORDS

Le catalogue

Contexte · La page d'accueil et le moteur de recherche interrogent le catalogue. Un livre est un **record** (immuable, accesseurs `titre()`, `auteur()`…). Toutes les méthodes s'écrivent avec un **Stream**, sans boucle for.

Ce que tu dois écrire

Q	Méthode	Comportement attendu
Q1	`livresDisponibles()`	Les livres avec au moins un exemplaire disponible, triés par titre.
Q2	`titresDe(auteur)`	Les titres d'un auteur, sans tenir compte des majuscules (« asimov » = « Asimov »).
Q3	`trouverParIsbn(isbn)`	Un `Optional<Livre>` : le livre s'il existe, sinon un Optional vide (jamais null).
Q4	`nombreDeLivresParAuteur()`	Une Map auteur → nombre de livres, auteurs triés par ordre alphabétique.
Q5	`lesPlusRecents(n)`	Les titres des n livres les plus récents, du plus récent au plus ancien.
Q6	`totalDisponibles()`	Le nombre total d'exemplaires disponibles dans la médiathèque.

Coups de pouce

- Trier : `sorted(Comparator.comparing(Livre::titre))` ; à l'envers : ajouter `.reversed()`.
- Regrouper et compter : `Collectors.groupingBy(clé, TreeMap::new, Collectors.counting())`.
- Garder les n premiers : `limit(n)`. Additionner : `mapToInt(...).sum()`.

Ce que les vérifications testent

Ton code doit faire passer chacune de ces vérifications :

Q	Vérification
Q1	4 livres disponibles
Q1	triés par titre : 1984 en premier, Les Robots en dernier
Q2	titres de « asimov » (minuscules acceptées)
Q2	auteur inconnu → liste vide
Q3	ISBN 978-03 → Dune
Q3	ISBN inconnu → Optional vide
Q4	{Asimov=2, Herbert=1, Orwell=2, Simmons=1}
Q5	les 2 plus récents : Hypérion puis Dune
Q6	6 exemplaires disponibles au total

Terminé quand · le fichier `Exercice2_Catalogue.java` affiche **Score : 9 / 9 Bravo !**

Le code de départ · `Exercice2_Catalogue.java`

Remplace chaque `throw new UnsupportedOperationException("À compléter");` par ton code. Le TODO juste au-dessus donne l'idée.

Exercice2_Catalogue.java · à compléter

```
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.TreeMap;
import java.util.function.BooleanSupplier;
import java.util.stream.Collectors;

/**
 * EXERCICE 2 · Le catalogue (Streams, Optional, records)
 *
 * Contexte : la page d'accueil et la recherche de la médiathèque interrogent le catalogue.
 * Toutes les méthodes s'écrivent avec un Stream, sans boucle for.
 *
 * Lancer : java Exercice2_Catalogue.java
 */
public class Exercice2_Catalogue {

    /** Un livre du catalogue (record : immuable, accesseurs titre(), auteur()…). */
    record Livre(String isbn, String titre, String auteur, int annee, int exemplaires, int
disponibles) {
        boolean estDisponible() { return disponibles > 0; }
    }

    static class Catalogue {
        private final List<Livre> livres;

        Catalogue(List<Livre> livres) {
            this.livres = List.copyOf(livres);
        }

        /** Q1. Les livres qui ont au moins un exemplaire disponible, triés par titre. */
        List<Livre> livresDisponibles() {
            // TODO : filter(Livre::estDisponible), sorted(Comparator.comparing(...)),
toList()
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q2. Les TITRES des livres d'un auteur, sans tenir compte des majuscules (
"asimov" = "Asimov"). */
        List<String> titresDe(String auteur) {
            // TODO : filter avec equalsIgnoreCase, puis map(Livre::titre)
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q3. Chercher un livre par ISBN. Absent → Optional vide (jamais null). */
        Optional<Livre> trouverParIsbn(String isbn) {
            // TODO : filter sur l'isbn puis findFirst()
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q4. Le nombre de livres par auteur, auteurs triés par ordre alphabétique. */
        Map<String, Long> nombreDeLivresParAuteur() {
            // TODO : collect(Collectors.groupingBy(Livre::auteur, TreeMap::new, Collectors.
counting()))
            throw new UnsupportedOperationException("À compléter");
        }
```

Exercice2_Catalogue.java · à compléter

```
        /** Q5. Les titres des n livres les plus récents (du plus récent au plus ancien). */
        List<String> lesPlusRecents(int n) {
            // TODO : sorted(Comparator.comparing(Livre::annee).reversed()), limit(n), map
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q6. Le nombre total d'exemplaires disponibles dans toute la médiathèque. */
        int totalDisponibles() {
            // TODO : mapToInt(Livre::disponibles).sum()
            throw new UnsupportedOperationException("À compléter");
        }
    }
```

3

RÈGLES R1 À R5 · EXCEPTIONS · JAVA.TIME · BIGDECIMAL

La politique d'emprunt

Contexte · C'est le **cœur métier** de la médiathèque, en Java pur. Quand une règle est violée, la politique lève une `RegleMetierException` (une RuntimeException, que l'API traduira en HTTP 409). Rappel : R1 · 3 emprunts en cours maximum. R2 · 21 jours. R3 · pas d'emprunt si inactif ou en retard. R4 · au moins un exemplaire disponible. R5 · 0,20 € par jour de retard, plafonné à 10 €.

Ce que tu dois écrire

Q	Méthode	Comportement attendu
Q1	`verifierEligibilite(...)`	Lever une RegleMetierException avec le bon message, dans cet ordre : inactif → « R3 · adhérent inactif », en retard → « R3 · emprunt en retard », 3 emprunts → « R1 · 3 emprunts en cours maximum », 0 exemplaire → « R4 · aucun exemplaire disponible ». Sinon, ne rien faire.
Q2	`dateRetourPrevue(date)`	Règle R2 : date d'emprunt + 21 jours (attention au changement de mois et d'année).
Q3	`estEnRetard(prevue, aujourdhui)`	En retard seulement **après** la date prévue : le jour même, pas de retard.
Q4	`calculerPenalite(prevue, effective)`	Règle R5 : 0,20 € par jour de retard, plafonné à 10 €, toujours avec 2 décimales (0.00, 1.60, 10.00).

Coups de pouce

- Lever une exception : `throw new RegleMetierException("...");` L'exception arrête la méthode.
- Dates : `plusDays(21)`, `isAfter(...)`, `ChronoUnit.DAYS.between(a, b)`.
- Argent : `multiply`, `min(PENALITE_MAX)`, `setScale(2, RoundingMode.HALF_UP)`. Jamais de double.

Ce que les vérifications testent

Ton code doit faire passer chacune de ces vérifications :

Q	Vérification
Q1	adhérent actif, 0 emprunt, livre disponible → autorisé
Q1	2 emprunts en cours → encore autorisé
Q1	R1 : 3 emprunts en cours → refusé
Q1	R3 : adhérent inactif → refusé
Q1	R3 : emprunt en retard → refusé
Q1	R4 : aucun exemplaire → refusé
Q1	priorité : inactif ET 3 emprunts → message R3
Q2	emprunt le 1er octobre → retour le 22 octobre
Q2	emprunt le 20 décembre → retour le 10 janvier (changement d'année)

Q	Vérification
Q3	le jour même → pas en retard
Q3	le lendemain → en retard
Q4	rendu à temps → 0.00
Q4	1 jour de retard → 0.20
Q4	8 jours de retard → 1.60
Q4	50 jours → 10.00 (exactement le plafond)
Q4	90 jours → 10.00 (plafonné)

Terminé quand · le fichier `Exercice3_PolitiqueEmprunt.java` affiche **Score : 16 / 16 Bravo !**

Le code de départ · `Exercice3_PolitiqueEmprunt.java`

Remplace chaque `throw new UnsupportedOperationException("À compléter");` par ton code. Le TODO juste au-dessus donne l'idée.

Exercice3_PolitiqueEmprunt.java · à compléter

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.function.BooleanSupplier;

/**
 * EXERCICE 3 · La politique d'emprunt (règles R1 à R5, exceptions métier, java.time,
BigDecimal)
 *
 * Contexte : c'est le cœur métier de la médiathèque. La classe PolitiqueEmprunt est en
Java pur :
 * pas de base de données, pas de Spring. Quand une règle est violée, elle lève une
 * RegleMetierException (que l'API traduira plus tard en HTTP 409).
 *
 * Rappel des règles :
 *   R1 · 3 emprunts en cours maximum par adhérent
 *   R2 · un emprunt dure 21 jours
 *   R3 · pas d'emprunt si l'adhérent est inactif ou a un emprunt en retard
 *   R4 · au moins un exemplaire disponible pour emprunter
 *   R5 · 0,20 € par jour de retard, plafonné à 10 €
 *
 * Lancer : java Exercice3_PolitiqueEmprunt.java
 */
public class Exercice3_PolitiqueEmprunt {

    /** Exception métier : une RuntimeException, donc pas de « throws » à écrire. */
    static class RegleMetierException extends RuntimeException {
        RegleMetierException(String message) { super(message); }
    }

    static class PolitiqueEmprunt {
        static final int MAX_EMPRUNTS_EN_COURS = 3;
        static final int DUREE_EMPRUNT_JOURS = 21;
        static final BigDecimal PENALITE_PAR_JOUR = new BigDecimal("0.20");
        static final BigDecimal PENALITE_MAX = new BigDecimal("10.00");

        /**
```

Exercice3_PolitiqueEmprunt.java · à compléter

```
         * Q1. Vérifier qu'un emprunt est autorisé. Lever RegleMetierException avec ces
messages,
         * dans cet ordre de priorité :
         *   inactif                     → "R3 · adhérent inactif"
         *   un emprunt en retard        → "R3 · emprunt en retard"
         *   déjà 3 emprunts en cours    → "R1 · 3 emprunts en cours maximum"
         *   aucun exemplaire disponible → "R4 · aucun exemplaire disponible"
         * Si tout va bien, la méthode ne fait rien.
         */
        void verifierEligibilite(boolean adherentActif, boolean aUnRetard, int
empruntsEnCours, int exemplairesDisponibles) {
            // TODO : une suite de if (...) throw new RegleMetierException("...");
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q2. Règle R2 : la date de retour prévue = date d'emprunt + 21 jours. */
        LocalDate dateRetourPrevue(LocalDate dateEmprunt) {
            // TODO : plusDays
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q3. Un emprunt est en retard si on est APRÈS la date de retour prévue (le jour
même : pas en retard). */
        boolean estEnRetard(LocalDate dateRetourPrevue, LocalDate aujourdhui) {
            // TODO : isAfter
            throw new UnsupportedOperationException("À compléter");
        }

        /**
         * Q4. Règle R5 : 0,20 € par jour de retard, plafonné à 10 €. Rendu à temps → 0.00.
         * Le résultat a toujours 2 décimales (ex. 1.60, 0.00, 10.00).
         */
        BigDecimal calculerPenalite(LocalDate dateRetourPrevue, LocalDate
dateRetourEffective) {
            // TODO : ChronoUnit.DAYS.between, multiply, min(PENALITE_MAX), setScale(2,
RoundingMode.HALF_UP)
            throw new UnsupportedOperationException("À compléter");
        }
    }
```

SYNTHÈSE · OBJETS · STREAMS · OPTIONAL · EXCEPTIONS

4 Le service d'emprunt

Contexte · Le service **orchestre** un emprunt de bout en bout : il retrouve le livre et l'adhérent, fait vérifier les règles par la PolitiqueEmprunt, met à jour le stock et enregistre l'emprunt. Les données sont en mémoire (des listes) : la base arrivera en séance 6. **Tout est fourni** (modèle, exceptions, politique) **sauf la classe ServiceEmprunt**.

Ce que tu dois écrire

Q	Méthode	Comportement attendu
Q1	`trouverLivre / trouverAdherent`	Retrouver par ISBN ou par email, sinon `RessourceIntrouvableException` (« Livre introuvable : … », « Adhérent introuvable : … ») → future erreur 404.
Q2	`empruntsEnCours(email)`	Les emprunts au statut EN_COURS de cet adhérent.
Q3	`aUnRetard(email, aujourdhui)`	Vrai si l'un de ses emprunts en cours est en retard à cette date.
Q4	`emprunter(isbn, email, date)`	Retrouver livre et adhérent, faire vérifier R1, R3 et R4 par la politique, retirer un exemplaire, créer l'emprunt (id 1, 2, 3…, retour prévu par R2), l'enregistrer et le renvoyer.
Q5	`rendre(id, date)`	Retrouver l'emprunt (sinon 404). S'il est déjà rendu → RegleMetierException « Emprunt déjà rendu ». Sinon calculer la pénalité (R5), clôturer l'emprunt, remettre l'exemplaire en stock.

Coups de pouce

- Chercher ou lever une exception : `.findFirst().orElseThrow(() -> new ...)`.
- « Au moins un » dans un Stream : `anyMatch(...)`.
- Réutilise tes propres méthodes : emprunter appelle trouverLivre, trouverAdherent, empruntsEnCours et aUnRetard.
- Certaines vérifications de Q2 et Q3 utilisent emprunter : elles passeront au vert après Q4.

Ce que les vérifications testent

Ton code doit faire passer chacune de ces vérifications :

Q	Vérification
Q1	trouver le livre 978-01 → Dune
Q1	ISBN inconnu → 404
Q1	email inconnu → 404
Q4	Alice emprunte Dune : emprunt n°1, retour le 22/10, stock 2 → 1
Q2	après 2 emprunts, Alice a 2 emprunts en cours
Q4	R3 : Bob est inactif → refusé
Q4	R4 : Fondation n'a qu'un exemplaire, 2e emprunt refusé
Q4	R1 : un 4e emprunt est refusé

Q	Vérification
Q3	le 30 octobre, un emprunt du 1er octobre est en retard
Q4	R3 : avec un retard, nouvel emprunt refusé
Q5	rendu le 30/10 : 8 jours de retard → 1.60 €, statut RENDU, stock remis
Q5	rendre deux fois → refusé
Q5	emprunt inconnu → 404

Terminé quand · le fichier `Exercice4_ServiceEmprunt.java` affiche **Score : 13 / 13 Bravo !**

Le code de départ · `Exercice4_ServiceEmprunt.java`

Remplace chaque `throw new UnsupportedOperationException("À compléter");` par ton code. Le TODO juste au-dessus donne l'idée.

Exercice4_ServiceEmprunt.java · à compléter

```
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;
import java.util.function.BooleanSupplier;

/**
 * EXERCICE 4 · Le service d'emprunt (synthèse : objets, Streams, Optional, exceptions,
règles)
 *
 * Contexte : le service orchestre un emprunt de bout en bout. Il retrouve le livre et
l'adhérent,
 * fait vérifier les règles par la PolitiqueEmprunt, met à jour le stock et enregistre
l'emprunt.
 * Les données sont gardées en mémoire (des listes) : la base de données arrivera en séance
6.
 *
 * Tout est FOURNI sauf la classe ServiceEmprunt, à compléter.
 *
 * Lancer : java Exercice4_ServiceEmprunt.java
 */
public class Exercice4_ServiceEmprunt {

    // ============================== FOURNI : exceptions métier
==============================
    static class RegleMetierException extends RuntimeException {             // → HTTP 409
        RegleMetierException(String message) { super(message); }
    }
    static class RessourceIntrouvableException extends RuntimeException {    // → HTTP 404
        RessourceIntrouvableException(String message) { super(message); }
    }

    // ============================== FOURNI : le modèle ==============================
    static class Livre {
        private final String isbn;
        private final String titre;
        private int disponibles;

        Livre(String isbn, String titre, int disponibles) {
            this.isbn = isbn; this.titre = titre; this.disponibles = disponibles;
```

Exercice4_ServiceEmprunt.java · à compléter

```
        }
        String getIsbn() { return isbn; }
        String getTitre() { return titre; }
        int getDisponibles() { return disponibles; }
        void emprunterExemplaire() { disponibles--; }
        void rendreExemplaire() { disponibles++; }
    }

    static class Adherent {
        private final String email;
        private final boolean actif;
        Adherent(String email, boolean actif) { this.email = email; this.actif = actif; }
        String getEmail() { return email; }
        boolean isActif() { return actif; }
    }

    enum StatutEmprunt { EN_COURS, RENDU }

    static class Emprunt {
        private final long id;
        private final Livre livre;
        private final Adherent adherent;
        private final LocalDate dateEmprunt;
        private final LocalDate dateRetourPrevue;
        private LocalDate dateRetourEffective;
        private BigDecimal penalite = BigDecimal.ZERO.setScale(2);
        private StatutEmprunt statut = StatutEmprunt.EN_COURS;

        Emprunt(long id, Livre livre, Adherent adherent, LocalDate dateEmprunt, LocalDate
dateRetourPrevue) {
            this.id = id; this.livre = livre; this.adherent = adherent;
            this.dateEmprunt = dateEmprunt; this.dateRetourPrevue = dateRetourPrevue;
        }
        void cloturer(LocalDate dateRetourEffective, BigDecimal penalite) {
            this.dateRetourEffective = dateRetourEffective;
            this.penalite = penalite;
            this.statut = StatutEmprunt.RENDU;
        }
        long getId() { return id; }
        Livre getLivre() { return livre; }
        Adherent getAdherent() { return adherent; }
        LocalDate getDateRetourPrevue() { return dateRetourPrevue; }
        BigDecimal getPenalite() { return penalite; }
        StatutEmprunt getStatut() { return statut; }
        boolean estEnCours() { return statut == StatutEmprunt.EN_COURS; }
    }

    // ============================== FOURNI : la politique (corrigé de l'exercice 3)
==============================
    static class PolitiqueEmprunt {
        void verifierEligibilite(boolean actif, boolean aUnRetard, int empruntsEnCours, int
disponibles) {
            if (!actif) throw new RegleMetierException("R3 · adhérent inactif");
            if (aUnRetard) throw new RegleMetierException("R3 · emprunt en retard");
            if (empruntsEnCours >= 3) throw new RegleMetierException("R1 · 3 emprunts en
cours maximum");
            if (disponibles <= 0) throw new RegleMetierException("R4 · aucun exemplaire
disponible");
        }
        LocalDate dateRetourPrevue(LocalDate dateEmprunt) { return dateEmprunt.plusDays(21);
 }
```

Exercice4_ServiceEmprunt.java · à compléter

```
        boolean estEnRetard(LocalDate prevue, LocalDate aujourdhui) { return aujourdhui.
isAfter(prevue); }
        BigDecimal calculerPenalite(LocalDate prevue, LocalDate effective) {
            long jours = ChronoUnit.DAYS.between(prevue, effective);
            if (jours <= 0) return BigDecimal.ZERO.setScale(2);
            return new BigDecimal("0.20").multiply(BigDecimal.valueOf(jours))
                    .min(new BigDecimal("10.00")).setScale(2, RoundingMode.HALF_UP);
        }
    }

    // ============================== À COMPLÉTER : le service
==============================
    static class ServiceEmprunt {
        private final List<Livre> livres;
        private final List<Adherent> adherents;
        private final List<Emprunt> emprunts = new ArrayList<>();
        private final PolitiqueEmprunt politique = new PolitiqueEmprunt();
        private long prochainId = 1;

        ServiceEmprunt(List<Livre> livres, List<Adherent> adherents) {
            this.livres = livres;
            this.adherents = adherents;
        }

        /** Q1. Retrouver un livre par ISBN, sinon RessourceIntrouvableException("Livre
introuvable : " + isbn). */
        Livre trouverLivre(String isbn) {
            // TODO : stream, filter, findFirst, orElseThrow(() -> new
RessourceIntrouvableException(...))
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q1 bis. Retrouver un adhérent par email, sinon RessourceIntrouvableException(
"Adhérent introuvable : " + email). */
        Adherent trouverAdherent(String email) {
            // TODO : même principe que trouverLivre
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q2. Les emprunts EN COURS d'un adhérent. */
        List<Emprunt> empruntsEnCours(String email) {
            // TODO : filter sur l'email de l'adhérent ET estEnCours()
            throw new UnsupportedOperationException("À compléter");
        }

        /** Q3. Vrai si l'un des emprunts en cours de l'adhérent est en retard à la date
donnée. */
        boolean aUnRetard(String email, LocalDate aujourdhui) {
            // TODO : anyMatch avec politique.estEnRetard(e.getDateRetourPrevue(),
aujourdhui)
            throw new UnsupportedOperationException("À compléter");
        }

        /**
         * Q4. Emprunter un livre :
         *  1. retrouver le livre et l'adhérent (404 s'ils n'existent pas)
         *  2. faire vérifier les règles par la politique (R1, R3, R4)
         *  3. retirer un exemplaire du stock
         *  4. créer l'emprunt (id = prochainId++, retour prévu par R2), l'ajouter à la
liste, le renvoyer
         */
        Emprunt emprunter(String isbn, String email, LocalDate dateEmprunt) {
```

Exercice4_ServiceEmprunt.java · à compléter

```
            // TODO : réutilise les méthodes Q1 à Q3, puis politique.verifierEligibilite(...
)
            throw new UnsupportedOperationException("À compléter");
        }

        /**
         * Q5. Rendre un livre :
         *  1. retrouver l'emprunt par id, sinon RessourceIntrouvableException("Emprunt
introuvable : " + id)
         *  2. s'il est déjà rendu → RegleMetierException("Emprunt déjà rendu")
         *  3. calculer la pénalité (R5), clôturer l'emprunt, remettre l'exemplaire en stock
         */
        Emprunt rendre(long empruntId, LocalDate dateRetour) {
            // TODO : findFirst + orElseThrow, puis if (!emprunt.estEnCours()) throw ...,
puis cloturer et rendreExemplaire
            throw new UnsupportedOperationException("À compléter");
        }
    }
```

Annexe · Le code des vérifications

Ce code se place **à la fin de chaque fichier**, juste après le code de départ (il ferme la classe avec la dernière accolade). Il est déjà présent dans les fichiers du zip : ne le modifie pas.

Exercice1_Adherent.java · fin du fichier

```
    // ============================== VÉRIFICATIONS (ne pas modifier)
==============================
    public static void main(String[] args) {
        LocalDate aujourdhui = LocalDate.of(2026, 10, 17);
        System.out.println("Exercice 1 · L'adhérent");

        verifier("Q1 · un adhérent valide est créé et actif", () -> {
            var a = new Adherent("Durand", "Alice", "alice@mail.fr", aujourdhui);
            return a.isActif() && a.getNom().equals("Durand");
        });
        verifier("Q1 · les espaces sont retirés et l'email est en minuscules", () -> {
            var a = new Adherent("  Durand ", " Alice", "  Alice.Durand@Mail.FR ",
aujourdhui);
            return a.getNom().equals("Durand") && a.getPrenom().equals("Alice")
                    && a.getEmail().equals("alice.durand@mail.fr");
        });
        verifierException("Q1 · nom vide refusé", IllegalArgumentException.class, "nom",
                () -> new Adherent("   ", "Alice", "alice@mail.fr", aujourdhui));
        verifierException("Q1 · prénom null refusé", IllegalArgumentException.class,
"prénom",
                () -> new Adherent("Durand", null, "alice@mail.fr", aujourdhui));
        verifierException("Q1 · email sans @ refusé", IllegalArgumentException.class,
"Email",
                () -> new Adherent("Durand", "Alice", "alice.mail.fr", aujourdhui));
        verifier("Q2 · nom complet = « Alice DURAND »",
                () -> new Adherent("Durand", "Alice", "alice@mail.fr", aujourdhui).
getNomComplet().equals("Alice DURAND"));
        verifier("Q3 · désactiver puis réactiver", () -> {
            var a = new Adherent("Durand", "Alice", "alice@mail.fr", aujourdhui);
            a.desactiver();
            boolean inactif = !a.isActif();
            a.reactiver();
            return inactif && a.isActif();
        });
        verifier("Q4 · inscrit il y a 6 jours → nouveau",
                () -> new Adherent("Durand", "Alice", "a@b.fr", aujourdhui.minusDays(6)).
estNouveau(aujourdhui));
        verifier("Q4 · inscrit il y a 7 jours → plus nouveau (limite exacte)",
                () -> !new Adherent("Durand", "Alice", "a@b.fr", aujourdhui.minusDays(7)).
estNouveau(aujourdhui));
        verifier("Q5 · même email → égaux, même hashCode", () -> {
            var a = new Adherent("Durand", "Alice", "alice@mail.fr", aujourdhui);
            var b = new Adherent("Dupont", "Alicia", "ALICE@mail.fr", aujourdhui.minusDays(
30));
            return a.equals(b) && a.hashCode() == b.hashCode();
        });
        verifier("Q5 · emails différents → pas égaux",
                () -> !new Adherent("Durand", "Alice", "alice@mail.fr", aujourdhui)
                        .equals(new Adherent("Durand", "Alice", "bob@mail.fr",
aujourdhui)));
        bilan();
    }

    // ------------------------------ outils de vérification ------------------------------
    static int reussis = 0, total = 0;
```

Exercice1_Adherent.java · fin du fichier

```
    static void verifier(String description, BooleanSupplier test) {
        total++;
        try {
            if (test.getAsBoolean()) { reussis++; System.out.println("  [OK]      " +
description); }
            else System.out.println("  [ERREUR]  " + description);
        } catch (UnsupportedOperationException e) {
            System.out.println("  [A FAIRE] " + description);
        } catch (Exception e) {
            System.out.println("  [ERREUR]  " + description + " → " + e);
        }
    }

    static void verifierException(String description, Class<? extends Exception> attendue,
String messageContient, Runnable code) {
        total++;
        try {
            code.run();
            System.out.println("  [ERREUR]  " + description + " → aucune exception levée");
        } catch (UnsupportedOperationException e) {
            System.out.println("  [A FAIRE] " + description);
        } catch (Exception e) {
            if (attendue.isInstance(e) && e.getMessage() != null && e.getMessage().contains(
messageContient)) {
                reussis++; System.out.println("  [OK]      " + description);
            } else {
                System.out.println("  [ERREUR]  " + description + " → " + e);
            }
        }
    }

    static void bilan() {
        System.out.println("Score : " + reussis + " / " + total + (reussis == total ? "
Bravo !" : ""));
    }
}
```

Exercice2_Catalogue.java · fin du fichier

```
    // ============================== VÉRIFICATIONS (ne pas modifier)
==============================
    public static void main(String[] args) {
        var catalogue = new Catalogue(List.of(
                new Livre("978-01", "Fondation", "Asimov", 1951, 3, 0),
                new Livre("978-02", "Les Robots", "Asimov", 1950, 2, 1),
                new Livre("978-03", "Dune", "Herbert", 1965, 4, 2),
                new Livre("978-04", "1984", "Orwell", 1949, 2, 2),
                new Livre("978-05", "La Ferme des animaux", "Orwell", 1945, 1, 0),
                new Livre("978-06", "Hypérion", "Simmons", 1989, 2, 1)));
        System.out.println("Exercice 2 · Le catalogue");

        verifier("Q1 · 4 livres disponibles", () -> catalogue.livresDisponibles().size() ==
4);
        verifier("Q1 · triés par titre : 1984 en premier, Les Robots en dernier", () -> {
            var l = catalogue.livresDisponibles();
            return l.get(0).titre().equals("1984") && l.get(l.size() - 1).titre().equals(
"Les Robots");
        });
        verifier("Q2 · titres de « asimov » (minuscules acceptées)",
                () -> catalogue.titresDe("asimov").equals(List.of("Fondation", "Les
Robots")));
```

Exercice2_Catalogue.java · fin du fichier

```
        verifier("Q2 · auteur inconnu → liste vide", () -> catalogue.titresDe("Tolkien").
isEmpty());
        verifier("Q3 · ISBN 978-03 → Dune",
                () -> catalogue.trouverParIsbn("978-03").map(Livre::titre).orElse("").
equals("Dune"));
        verifier("Q3 · ISBN inconnu → Optional vide", () -> catalogue.trouverParIsbn("000").
isEmpty());
        verifier("Q4 · {Asimov=2, Herbert=1, Orwell=2, Simmons=1}",
                () -> catalogue.nombreDeLivresParAuteur().toString().equals("{Asimov=2,
Herbert=1, Orwell=2, Simmons=1}"));
        verifier("Q5 · les 2 plus récents : Hypérion puis Dune",
                () -> catalogue.lesPlusRecents(2).equals(List.of("Hypérion", "Dune")));
        verifier("Q6 · 6 exemplaires disponibles au total", () -> catalogue.
totalDisponibles() == 6);
        bilan();
    }

    // ------------------------------ outils de vérification ------------------------------
    static int reussis = 0, total = 0;

    static void verifier(String description, BooleanSupplier test) {
        total++;
        try {
            if (test.getAsBoolean()) { reussis++; System.out.println("  [OK]      " +
description); }
            else System.out.println("  [ERREUR]  " + description);
        } catch (UnsupportedOperationException e) {
            System.out.println("  [A FAIRE] " + description);
        } catch (Exception e) {
            System.out.println("  [ERREUR]  " + description + " → " + e);
        }
    }

    static void verifierException(String description, Class<? extends Exception> attendue,
String messageContient, Runnable code) {
        total++;
        try {
            code.run();
            System.out.println("  [ERREUR]  " + description + " → aucune exception levée");
        } catch (UnsupportedOperationException e) {
            System.out.println("  [A FAIRE] " + description);
        } catch (Exception e) {
            if (attendue.isInstance(e) && e.getMessage() != null && e.getMessage().contains(
messageContient)) {
                reussis++; System.out.println("  [OK]      " + description);
            } else {
                System.out.println("  [ERREUR]  " + description + " → " + e);
            }
        }
    }

    static void bilan() {
        System.out.println("Score : " + reussis + " / " + total + (reussis == total ? "
Bravo !" : ""));
    }
}
```

Exercice3_PolitiqueEmprunt.java · fin du fichier

```
    // ============================== VÉRIFICATIONS (ne pas modifier)
==============================
    public static void main(String[] args) {
        var p = new PolitiqueEmprunt();
        LocalDate prevu = LocalDate.of(2026, 10, 22);
        System.out.println("Exercice 3 · La politique d'emprunt");

        verifier("Q1 · adhérent actif, 0 emprunt, livre disponible → autorisé", () -> {
            p.verifierEligibilite(true, false, 0, 2);
            return true;
        });
        verifier("Q1 · 2 emprunts en cours → encore autorisé", () -> {
            p.verifierEligibilite(true, false, 2, 1);
            return true;
        });
        verifierException("Q1 · R1 : 3 emprunts en cours → refusé", RegleMetierException.
class, "R1",
                () -> p.verifierEligibilite(true, false, 3, 1));
        verifierException("Q1 · R3 : adhérent inactif → refusé", RegleMetierException.class,
 "inactif",
                () -> p.verifierEligibilite(false, false, 0, 1));
        verifierException("Q1 · R3 : emprunt en retard → refusé", RegleMetierException.
class, "retard",
                () -> p.verifierEligibilite(true, true, 0, 1));
        verifierException("Q1 · R4 : aucun exemplaire → refusé", RegleMetierException.class,
 "R4",
                () -> p.verifierEligibilite(true, false, 0, 0));
        verifierException("Q1 · priorité : inactif ET 3 emprunts → message R3",
RegleMetierException.class, "R3",
                () -> p.verifierEligibilite(false, false, 3, 0));
        verifier("Q2 · emprunt le 1er octobre → retour le 22 octobre",
                () -> p.dateRetourPrevue(LocalDate.of(2026, 10, 1)).equals(prevu));
        verifier("Q2 · emprunt le 20 décembre → retour le 10 janvier (changement d'année)",
                () -> p.dateRetourPrevue(LocalDate.of(2026, 12, 20)).equals(LocalDate.of(
2027, 1, 10)));
        verifier("Q3 · le jour même → pas en retard", () -> !p.estEnRetard(prevu, prevu));
        verifier("Q3 · le lendemain → en retard", () -> p.estEnRetard(prevu, prevu.plusDays(
1)));
        verifier("Q4 · rendu à temps → 0.00", () -> p.calculerPenalite(prevu, prevu.
minusDays(3)).toString().equals("0.00"));
        verifier("Q4 · 1 jour de retard → 0.20", () -> p.calculerPenalite(prevu, prevu.
plusDays(1)).toString().equals("0.20"));
        verifier("Q4 · 8 jours de retard → 1.60", () -> p.calculerPenalite(prevu, prevu.
plusDays(8)).toString().equals("1.60"));
        verifier("Q4 · 50 jours → 10.00 (exactement le plafond)", () -> p.calculerPenalite(
prevu, prevu.plusDays(50)).toString().equals("10.00"));
        verifier("Q4 · 90 jours → 10.00 (plafonné)", () -> p.calculerPenalite(prevu, prevu.
plusDays(90)).toString().equals("10.00"));
        bilan();
    }

    // ------------------------------ outils de vérification ------------------------------
    static int reussis = 0, total = 0;

    static void verifier(String description, BooleanSupplier test) {
        total++;
        try {
            if (test.getAsBoolean()) { reussis++; System.out.println("  [OK]      " +
description); }
            else System.out.println("  [ERREUR]  " + description);
        } catch (UnsupportedOperationException e) {
            System.out.println("  [A FAIRE] " + description);
```

Exercice3_PolitiqueEmprunt.java · fin du fichier

```
        } catch (Exception e) {
            System.out.println("  [ERREUR]  " + description + " → " + e);
        }
    }

    static void verifierException(String description, Class<? extends Exception> attendue,
String messageContient, Runnable code) {
        total++;
        try {
            code.run();
            System.out.println("  [ERREUR]  " + description + " → aucune exception levée");
        } catch (UnsupportedOperationException e) {
            System.out.println("  [A FAIRE] " + description);
        } catch (Exception e) {
            if (attendue.isInstance(e) && e.getMessage() != null && e.getMessage().contains(
messageContient)) {
                reussis++; System.out.println("  [OK]      " + description);
            } else {
                System.out.println("  [ERREUR]  " + description + " → " + e);
            }
        }
    }

    static void bilan() {
        System.out.println("Score : " + reussis + " / " + total + (reussis == total ? "
Bravo !" : ""));
    }
}
```

Exercice4_ServiceEmprunt.java · fin du fichier

```
    // ============================== VÉRIFICATIONS (ne pas modifier)
==============================
    static ServiceEmprunt nouveauService() {
        return new ServiceEmprunt(
                List.of(new Livre("978-01", "Dune", 2), new Livre("978-02", "Fondation", 1),
                        new Livre("978-03", "1984", 3), new Livre("978-04", "Hypérion", 1)),
                List.of(new Adherent("alice@mail.fr", true), new Adherent("bob@mail.fr",
false)));
    }

    public static void main(String[] args) {
        LocalDate j1 = LocalDate.of(2026, 10, 1);
        System.out.println("Exercice 4 · Le service d'emprunt");

        verifier("Q1 · trouver le livre 978-01 → Dune", () -> nouveauService().trouverLivre(
"978-01").getTitre().equals("Dune"));
        verifierException("Q1 · ISBN inconnu → 404", RessourceIntrouvableException.class,
"Livre introuvable",
                () -> nouveauService().trouverLivre("000"));
        verifierException("Q1 · email inconnu → 404", RessourceIntrouvableException.class,
"Adhérent introuvable",
                () -> nouveauService().trouverAdherent("zoe@mail.fr"));
        verifier("Q4 · Alice emprunte Dune : emprunt n°1, retour le 22/10, stock 2 → 1", ()
-> {
            var s = nouveauService();
            var e = s.emprunter("978-01", "alice@mail.fr", j1);
            return e.getId() == 1 && e.getDateRetourPrevue().equals(LocalDate.of(2026, 10,
22))
                    && s.trouverLivre("978-01").getDisponibles() == 1;
        });
        verifier("Q2 · après 2 emprunts, Alice a 2 emprunts en cours", () -> {
```

Exercice4_ServiceEmprunt.java · fin du fichier

```
            var s = nouveauService();
            s.emprunter("978-01", "alice@mail.fr", j1);
            s.emprunter("978-03", "alice@mail.fr", j1);
            return s.empruntsEnCours("alice@mail.fr").size() == 2;
        });
        verifierException("Q4 · R3 : Bob est inactif → refusé", RegleMetierException.class,
"inactif",
                () -> nouveauService().emprunter("978-01", "bob@mail.fr", j1));
        verifierException("Q4 · R4 : Fondation n'a qu'un exemplaire, 2e emprunt refusé",
RegleMetierException.class, "R4", () -> {
            var s = nouveauService();
            s.emprunter("978-02", "alice@mail.fr", j1);
            s.emprunter("978-02", "alice@mail.fr", j1);
        });
        verifierException("Q4 · R1 : un 4e emprunt est refusé", RegleMetierException.class,
"R1", () -> {
            var s = nouveauService();
            s.emprunter("978-01", "alice@mail.fr", j1);
            s.emprunter("978-02", "alice@mail.fr", j1);
            s.emprunter("978-03", "alice@mail.fr", j1);
            s.emprunter("978-04", "alice@mail.fr", j1);
        });
        verifier("Q3 · le 30 octobre, un emprunt du 1er octobre est en retard", () -> {
            var s = nouveauService();
            s.emprunter("978-01", "alice@mail.fr", j1);
            return !s.aUnRetard("alice@mail.fr", LocalDate.of(2026, 10, 22))
                    && s.aUnRetard("alice@mail.fr", LocalDate.of(2026, 10, 30));
        });
        verifierException("Q4 · R3 : avec un retard, nouvel emprunt refusé",
RegleMetierException.class, "retard", () -> {
            var s = nouveauService();
            s.emprunter("978-01", "alice@mail.fr", j1);
            s.emprunter("978-03", "alice@mail.fr", LocalDate.of(2026, 10, 30));
        });
        verifier("Q5 · rendu le 30/10 : 8 jours de retard → 1.60 €, statut RENDU, stock
remis", () -> {
            var s = nouveauService();
            var e = s.emprunter("978-01", "alice@mail.fr", j1);
            var rendu = s.rendre(e.getId(), LocalDate.of(2026, 10, 30));
            return rendu.getPenalite().toString().equals("1.60") && rendu.getStatut() ==
StatutEmprunt.RENDU
                    && s.trouverLivre("978-01").getDisponibles() == 2 && s.empruntsEnCours(
"alice@mail.fr").isEmpty();
        });
        verifierException("Q5 · rendre deux fois → refusé", RegleMetierException.class,
"déjà rendu", () -> {
            var s = nouveauService();
            var e = s.emprunter("978-01", "alice@mail.fr", j1);
            s.rendre(e.getId(), LocalDate.of(2026, 10, 10));
            s.rendre(e.getId(), LocalDate.of(2026, 10, 11));
        });
        verifierException("Q5 · emprunt inconnu → 404", RessourceIntrouvableException.class,
 "Emprunt introuvable",
                () -> nouveauService().rendre(99, j1));
        bilan();
    }

    // ------------------------------ outils de vérification ------------------------------
    static int reussis = 0, total = 0;

    static void verifier(String description, BooleanSupplier test) {
        total++;
```

Exercice4_ServiceEmprunt.java · fin du fichier

```
        try {
            if (test.getAsBoolean()) { reussis++; System.out.println("  [OK]      " +
description); }
            else System.out.println("  [ERREUR]  " + description);
        } catch (UnsupportedOperationException e) {
            System.out.println("  [A FAIRE] " + description);
        } catch (Exception e) {
            System.out.println("  [ERREUR]  " + description + " → " + e);
        }
    }

    static void verifierException(String description, Class<? extends Exception> attendue,
String messageContient, Runnable code) {
        total++;
        try {
            code.run();
            System.out.println("  [ERREUR]  " + description + " → aucune exception levée");
        } catch (UnsupportedOperationException e) {
            System.out.println("  [A FAIRE] " + description);
        } catch (Exception e) {
            if (attendue.isInstance(e) && e.getMessage() != null && e.getMessage().contains(
messageContient)) {
                reussis++; System.out.println("  [OK]      " + description);
            } else {
                System.out.println("  [ERREUR]  " + description + " → " + e);
            }
        }
    }

    static void bilan() {
        System.out.println("Score : " + reussis + " / " + total + (reussis == total ? "
Bravo !" : ""));
    }
}
```